

QTC — Current Implementation, Phases, Roadmaps & Upgrade Plans

Prepared October 2, 2026

Prepared: October 2, 2026

Executive Summary

QTC (Quantum Coin) is a Bitcoin-derived cryptocurrency project focused on replacing classical public-key cryptography with post-quantum cryptography while retaining a familiar UTXO blockchain architecture.

The current PQNativeConcept repository contains substantial QTC-specific work: Kyber1024/ML-KEM-1024, Dilithium3/ML-DSA-65, SHA3-256/SHA3-512 work, qRandomX, RandomX/Cuckoo research, PQ Noise networking, PQ-HD wallets, qtc1 Bech32m addresses, witness hashing/compression research, script protections, mining tools, test suites, audits, and deployment documentation.

The repository itself says it is still under active development and is not production-ready. Its audit identifies important integration and build gaps even though major cryptographic components are documented as implemented.

The recommended new architecture is:

CLEAN BITCOINII FOUNDATION

+

SELECTED QTC PQ TECHNOLOGY

+

CANONICAL PQ WALLET

+

EXPLICIT UPGRADE/VERSIONING SYSTEM

The first working QTC should be intentionally simpler:

- 21,000,000 QTC maximum supply
- approximately 10-minute blocks
- initially target 32 MB blocks, subject to stress/propagation testing
- SHA-256 Proof of Work so existing SHA-256 mining hardware can potentially mine QTC
- ML-DSA-65/Dilithium3-class transaction signatures
- ML-KEM-1024/Kyber1024 where KEM functionality is needed
- one canonical qtc1 Bech32m address scheme
- native PQ wallet support
- BitcoinII-derived UTXO/node/RPC foundation
- future migration points for PoW, signatures, KEMs, addresses and networking

The previous qRandomX design should be preserved as a future mining research/upgrade track rather than being required for the first launch.

1. Current QTC Architecture

The original design can be summarized as:

QTC Core

- Mining: qRandomX → Kyber1024 + RandomX + Cuckoo + BLAKE3/SHA3
- Network: PQ Noise → Kyber1024 + Dilithium3 + SHA3-512 + ChaCha20-Poly1305
- Wallet: PQ-HD → Kyber1024 + Dilithium3 + SHA3-derived addresses
- Consensus: PQ transaction validation, witness processing and script protections

The technical architecture document explicitly describes qRandomX, PQ Noise, PQ-HD wallet technology and PQ validation as major components.

2. Current Cryptography

ML-KEM-1024 / Kyber1024

The audit identifies the Kyber implementation under `src/crypto/kyber/`.

Documented properties include the FIPS-203 parameter set, 1568-byte public keys, 1568-byte ciphertexts, 32-byte shared secrets, Fujisaki-Okamoto protection, NTT operations, polynomial operations and compression.

QTC role:

- P2P key establishment
- encrypted session establishment
- other key-establishment applications

ML-KEM is not a digital signature algorithm.

ML-DSA-65 / Dilithium3

The audit identifies the Dilithium implementation under `src/crypto/dilithium/`.

Documented properties include 1952-byte public keys, 4000-byte secret keys and 3293-byte signatures, with Fiat-Shamir, rejection sampling, NTT and packing/unpacking.

QTC role:

- transaction authorization
- wallet signing
- node identity where required

The standardized name should be ML-DSA-65; Dilithium3 can remain as a historical compatibility name.

Hashing

The project contains SHA3-256, SHA3-512, SHAKE/CSHAKE-related work and BLAKE3/SHA3 mining finalization.

For the new chain, every consensus hash construction should be explicitly frozen and test-vectorized. Avoid unnecessary competing consensus hash constructions.

3. Address Architecture

The current repository documents two different address-generation methods.

Primary/Q4:

Dilithium public key → SHA3-256 → first 20 bytes → Bech32m → qtc address, documented as witness version 1.

PQ-HD:

Kyber shared secret + Dilithium public key → SHA3-512 derivation → SHA3-256 → first 20 bytes → Bech32m → qtc address, documented as witness version 2.

This is an important unresolved protocol issue.

For QTC v1, freeze exactly ONE canonical address construction:

PQ key material → canonical commitment → canonical witness program → Bech32m → qtc1...

Use a separate testnet HRP such as tqtc1....

QTC-Wallet and QTC Core must generate byte-for-byte identical addresses from permanent test vectors.

4. Witness Hash / Compression

The repository contains `qtc_witness_hash.h` and documents SHA3-256 of witness data followed by extraction of the first 20 bytes.

The documentation describes reducing a 3293-byte Dilithium signature to a 20-byte witness hash.

This should remain research until its exact security and consensus semantics are independently validated.

Before using it in mainnet consensus, define:

- exactly what is committed
- exactly what is revealed
- signature verification rules
- collision/preimage assumptions
- malleability behavior
- recovery behavior
- permanent test vectors

5. qRandomX

The current mining architecture is substantially more complex than Bitcoin-style SHA-256:

Kyber1024 epoch/challenge

→ RandomX VM

→ Cuckoo Cycle

→ BLAKE3

→ SHA3-512

→ final 256-bit result

The audit reports qRandomX components including Kyber1024, RandomX/JIT, Cuckoo Cycle and hybrid BLAKE3/SHA3 finalization.

The repository also contains production mining code, one-click mining, standalone mining and desktop mining tools.

New decision

Do not make qRandomX the first-launch consensus algorithm.

Preserve it as QTC Future Mining Research.

6. New QTC v1 Mining Plan

QTC v1 should use:

Block header

→ SHA-256

→ SHA-256

→ target comparison

This gives:

- a much smaller consensus surface
- mature mining software/hardware
- potential compatibility with existing SHA-256 ASICs
- fewer dependencies
- easier debugging
- simpler launch testing

This does not mean SHA-256 itself is post-quantum. Mining security and transaction ownership are separate layers.

7. Monetary Policy

Target maximum supply:

21,000,000 QTC

Use a Bitcoin-like subsidy/halving model, but independently specify and test:

- initial subsidy
- halving interval
- minimum unit
- coinbase maturity
- rounding
- final-supply enforcement

Automated consensus tests should prove that the chain can never exceed the intended monetary limit.

8. Block Time

Target:

approximately 10 minutes per block.

Use an independently specified difficulty algorithm.

Do not port proprietary ShockWave difficulty code from BitcoinII. The current BitcoinII repository contains an explicit notice that the original ShockWave implementation is proprietary and cannot be incorporated into another blockchain without permission.

9. Block Size

Initial engineering target:

32 MB.

This is a target to test, not merely a constant.

Test:

- serialization
- validation time
- signature verification load
- transaction count
- mempool limits
- relay
- orphan behavior
- disk/database performance
- reindexing
- synchronization
- worst-case PQ signature workloads
- bandwidth/propagation

Large PQ signatures make this especially important.

10. PQ P2P

The current repository documents:

NoisePQ_KYBER1024_DILITHIUM3_SHA3-512_CHACHA20-POLY1305

Features described include:

- Kyber1024 key exchange
- Dilithium3 authentication
- HKDF-SHA3-512
- ChaCha20-Poly1305
- ephemeral keys
- forward secrecy
- session resumption
- rekeying

However, the comprehensive audit identifies integration gaps in the old implementation:

- pq_noise.cpp not integrated into the build
- no net_processing integration
- missing/unfinished DoS protections in the audited state
- missing unit/interop/fuzz testing
- telemetry not fully connected

Roadmap:

v1: stable BitcoinII-derived P2P

v1.x: PQ handshake + fuzzing + interop + DoS protection + downgrade resistance

v2+: full PQ transport deployment after validation

11. PQ-HD Wallet

The old QTC repository contains a PQ-HD wallet architecture using deterministic PQ keys, Kyber1024, Dilithium3 and SHA3-derived material.

This is valuable work to preserve, but QTC v1 should freeze one canonical derivation/address model instead of supporting several competing historical models.

12. Current QTC-Wallet Upgrade

The current QTC-Wallet repository has moved toward a native C/C++ liboqs wrapper.

Documented scripts:

- qti2.js — standard PQ wallet
- qti3.js — PQ-HD wallet
- qti3_hd.js — hierarchical deterministic wallet
- oqs_wallet_cli — native deterministic key-generation wrapper

The wrapper supports deterministic:

- ML-KEM-1024
- ML-DSA-65

It documents Windows MSVC support and Linux/macOS compiler support.

This wallet should become the reference implementation for:

- key generation
- deterministic derivation
- serialization
- address generation
- signing

- recovery
- compatibility test vectors

Node.js should not be a consensus dependency.

13. Wallet Architecture

Recommended:

QTC GUI/CLI

→ QTC Wallet

→ native QTC cryptography

→ ML-DSA / ML-KEM

→ QTC Core RPC

→ QTC Node

The external QTC-Wallet can remain a reference/compatibility implementation.

14. Build Problems in PQNativeConcept

The audit identifies several categories.

Historical API mismatches included older calls such as:

`qtc_dilithium::ml_dsa65::keygen()`

`qtc_dilithium::ml_dsa65::sign()`

`qtc_dilithium::ml_dsa65::verify()`

versus:

`qtc_dilithium::GenerateKeys()`

`qtc_dilithium::Sign()`

`qtc_dilithium::Verify()`

The repository later documents these calling-code mismatches as fixed, so they should be checked against the exact current commit rather than assumed to remain current.

Other documented problems:

- BITCOIN_* vs QTC_* build-variable inconsistencies
- tests including .cpp instead of headers
- RandomX dependency integration
- PQ P2P missing from build
- dependency/linking issues
- CSHAKE integration work

The build troubleshooting guide also documents dependency and compiler issues, RandomX setup, PQ include paths and debugging methods.

15. Can the Old QTC Attempt Be Repaired?

Yes, the documented problems are largely engineering/integration problems.

However, repair and launch are different decisions.

The old project combines:

PQ crypto + wallet + script changes + new mining + new P2P + witness changes + build changes + dependencies.

That creates a very large debugging surface.

Recommended role:

PQNativeConcept = QTC Research / Reference / Archive

New QTC-Core = production blockchain foundation.

16. New Repository Strategy

QTC-Core

Start from the clean BitcoinII-Core build.

Responsibilities:

- blockchain
- UTXO
- consensus
- block validation
- SHA-256 mining
- RPC
- P2P
- native PQ transaction validation
- wallet integration
- tests

QTC-Wallet

Responsibilities:

- PQ key generation
- deterministic derivation
- qtc1 addresses
- signing
- recovery
- test vectors
- compatibility

PQNativeConcept

Responsibilities:

- research archive
- qRandomX
- RandomX/Cuckoo experiments
- PQ P2P research
- old wallet implementations
- witness compression research
- audits
- architecture documentation
- historical mining tools

17. Keep / Modify / Port / Defer

KEEP from BitcoinII:

- UTXO
- validation
- database
- transaction framework
- RPC
- P2P foundation
- test framework
- SHA-256 PoW foundation

MODIFY:

- network parameters
- chain parameters
- genesis
- subsidy
- block limits
- addresses
- transaction authorization
- wallet interfaces
- node names
- ports
- data directories

PORT selectively from QTC:

- ML-KEM
- ML-DSA
- deterministic PQ derivation
- qtc1 address logic
- test vectors
- validated hash utilities
- selected PQ P2P design

DEFER:

- qRandomX
- RandomX dependency
- Cuckoo consensus
- full PQ P2P
- OP_CAT experiments
- complex witness compression
- multiple address versions
- browser-only cryptography

18. Complete Development Phases

Phase 0 — Freeze old work

Create a permanent archive/tag containing source, wallet, documentation, audits, test vectors, mining research, PQ P2P and whitepaper.

Exit: reproducible historical state.

Phase 1 — Clean BitcoinII baseline

Verify:

node, CLI, wallet, RPC, tests, regtest, database and P2P.

Do not add PQ changes.

Exit: clean reproducible baseline.

Phase 2 — QTC identity

Change binary names, configuration, data directories, P2P magic, ports, RPC identity, network IDs and chain identity.

Exit: QTC cannot accidentally join BitcoinII.

Phase 3 — Monetary policy

Implement and test 21M supply, subsidy, halving, block interval, coinbase maturity, minimum unit and rounding.

Exit: automated supply tests pass.

Phase 4 — Genesis

Freeze genesis timestamp, message, transaction, Merkle root, nonce, target and genesis hash.

Exit: every QTC node agrees on genesis.

Phase 5 — Larger blocks

Implement and stress-test the 32 MB target.

Exit: large blocks propagate, validate, reindex and synchronize safely.

Phase 6 — SHA-256 mining

Implement block templates, coinbase, target validation, difficulty, mining RPC and block submission.

Exit: standard SHA-256 mining software can mine a valid QTC block.

Phase 7 — Native PQ crypto

Create a clean native PQ layer:

src/crypto/pq/mlkem

src/crypto/pq/mldsa

src/crypto/pq/ hashing

src/crypto/pq/serialization

src/crypto/pq/test_vectors

Exit: deterministic ML-KEM/ML-DSA vectors pass.

Phase 8 — PQ transaction type

Freeze public-key encoding, signature encoding, sighash, transaction serialization, witness structure, validation, fees and malleability behavior.

Exit: independent QTC nodes verify the same transaction.

Phase 9 — qtc1 address

Freeze one address method and create permanent seed/key/public-key/address vectors.

Exit: QTC-Wallet and QTC Core match byte-for-byte.

Phase 10 — Wallet integration

Integrate native wallet support.

Exit: create → sign → broadcast → receive → spend works.

Phase 11 — Testnet

Test wallet creation/recovery, transactions, invalid signatures, malformed keys, blocks, mining, reorgs, synchronization, fees, coinbase maturity and RPC.

Exit: stable multi-node testnet.

Phase 12 — PQ P2P

Port PQ Noise and add version negotiation, downgrade resistance, DoS protection, replay protection, fuzzing, interop and telemetry.

Exit: PQ P2P independently tested.

Phase 13 — Mainnet candidate

Freeze consensus, genesis, address format, signature algorithm, KEM use, block size, block time, difficulty, subsidy, ports, wallet format and RPC.

Then perform reproducible builds, code review, fuzzing, soak testing and wallet recovery testing.

19. Roadmap After v1

QTC v1 — Launch

21M

~10-minute blocks

32 MB target

SHA-256 PoW

ML-DSA transaction signatures

ML-KEM where required

qtc1

PQ wallet

BitcoinII-derived node

QTC v1.x — Hardening

- PQ P2P
- wallet improvements
- hardware wallet support
- explorer/indexer
- fuzzing
- performance improvements

- monitoring
- recovery improvements

QTC v2 — PoW Evolution

Research candidates:

- qRandomX
- RandomX
- Cuckoo
- memory-hard hybrids
- epoch-seeded challenges
- other quantum-aware PoW

Do not hard-code a future PoW until research and testing justify it.

QTC v3+ — Cryptographic Migration

Potential:

- new PQ signatures
- new KEMs
- address versions
- witness versions
- new signature encodings
- hybrid migration if standards/threats require it

20. Upgrade Architecture

Every major subsystem should be versionable:

QTC Protocol

- Transaction Version
- Signature Algorithm ID
- KEM Algorithm ID
- Address Version
- Witness Version
- PoW Algorithm ID
- Block Version
- Network Protocol Version

This permits QTC v1 to evolve without redesigning the entire chain.

21. Quantum-Security Model

Separate the security layers:

Coin ownership → PQ signatures → ML-DSA

Encrypted sessions → PQ KEM → ML-KEM

Hashing → specified hash primitives

Mining → SHA-256 v1 → future migration capability

This means QTC can have PQ-resistant transaction ownership while intentionally retaining SHA-256 mining during its first phase.

Avoid claiming absolute or permanent “100% quantum safe” status. A technically stronger description is:

“QTC is a Bitcoin-derived cryptocurrency designed around post-quantum-resistant transaction authorization and an upgradeable cryptographic architecture.”

22. Most Valuable Existing QTC Assets

Preserve:

1. ML-KEM/Kyber implementation
2. ML-DSA/Dilithium implementation
3. PQ key generation
4. PQ-HD derivation
5. qtc1 address work
6. witness hashing research
7. PQ test vectors
8. Kyber tests
9. Dilithium tests
10. PQ P2P design
11. PQ Noise design
12. script protection research
13. qRandomX
14. RandomX/Cuckoo research
15. mining tools
16. deployment guides
17. audits
18. whitepaper
19. security analysis
20. QTC-Wallet

23. Main Risks to Resolve

Consensus:

- freeze address construction
- freeze witness model
- freeze signature serialization
- independently specify difficulty
- test 32 MB blocks

Cryptography:

- validate implementations
- create deterministic test vectors
- correctly implement SHAKE/CSHAKE dependencies
- avoid undocumented cryptographic shortcuts
- provide future migration paths

Wallet:

- eliminate competing address schemes
- canonicalize derivation
- make native and reference wallet outputs identical
- test recovery
- canonicalize key serialization

Networking:

- complete PQ handshake integration
- DoS protection
- fuzzing
- downgrade resistance
- interoperability

Mining:

- keep qRandomX out of the first launch
- avoid RandomX dependency at genesis
- avoid Cuckoo consensus complexity at genesis
- use SHA-256 v1

24. Immediate Development Order

1. Clean BitcoinII build
2. QTC identity
3. 21M monetary policy
4. Genesis
5. 32 MB block design
6. SHA-256 mining
7. Native ML-DSA
8. Canonical PQ transaction
9. Canonical qtc1 address
10. QTC-Wallet compatibility
11. Multi-node testnet
12. PQ P2P
13. Mainnet candidate
14. Future qRandomX/PoW research

25. Final Recommendation

Do not throw away the previous QTC implementation.

Separate it into reusable technology and research.

The clean BitcoinII fork becomes the stable skeleton.

QTC-Wallet becomes the cryptographic/address reference.

PQNativeConcept becomes the QTC research archive and source library.

The first launch should be:

21M

+
~10-minute blocks
+
larger blocks
+
SHA-256 PoW
+
PQ transaction ownership
+
canonical qtc1 addresses
+
native PQ wallet
+
upgradeable protocol

Then the more ambitious components can arrive incrementally:

PQ P2P → advanced wallet features → qRandomX/PoW research → future cryptographic migrations.

Appendix — Production Checklist

- ☐ reproducible builds
- ☐ frozen consensus specification
- ☐ frozen genesis
- ☐ 21M supply tests
- ☐ subsidy/halving tests
- ☐ difficulty tests
- ☐ 32 MB stress tests
- ☐ ML-DSA test vectors
- ☐ ML-KEM test vectors
- ☐ address test vectors
- ☐ wallet recovery tests
- ☐ cross-wallet compatibility
- ☐ invalid-signature tests
- ☐ malformed-key tests
- ☐ fuzzing
- ☐ multi-node testnet
- ☐ reorg tests
- ☐ large-block propagation tests
- ☐ P2P DoS tests
- ☐ PQ P2P interop tests if enabled
- ☐ independent code review
- ☐ cryptographic review
- ☐ economic/DoS analysis

- [] documented upgrade mechanism
- [] mainnet procedure

Appendix — Source Material Reviewed

- akdavis83/PQNativeConcept
- QTC_Comprehensive_Audit_Report.md
- QTC_Technical_Architecture.md
- BUILD_TROUBLESHOOTING.md
- QTC_FIX_TASK_LIST.md
- QTC_PQ_HD_WALLET_README.md
- QTC_PQ_P2P_NETWORKING_README.md
- current akdavis83/QTC-Wallet
- current BitcoinII-Core

Appendix — BitcoinII Licensing Note

The current BitcoinII-Core repository explicitly states that its original ShockWave difficulty implementation is proprietary and may be viewed/reviewed/tested but may not be incorporated, ported, redistributed or used as the basis for another production blockchain without permission.

QTC should therefore implement its own independently specified, appropriately licensed difficulty algorithm.